Computational Methods PS2

Alastair Watt

March 2026

A.)

Write a Matlab program that solves

$$\alpha \int_0^1 \exp(\alpha x - x^2/2) dx = 1$$

for α, using 20 node Gauss-Legendre quadrature. Explain the basic principles behind this quadrature method.

Answer:

The basic principles behind the Gauss-Legendre Quadrature:

- Unlike other quadrature methods, the Newton-Cotes methods, the Gauss-Legendre method optimally chooses nodes (x_i) and weights (w_i) to exactly integrate as high-degree polynomials as possible.
- An n-point Gauss–Legendre rule operates on $[-1, 1]$ is exact for all polynomials up to 2n-1. Thankfully there are methods to change an interval $[a, b]$ to $[-1, 1]$ as we will need to do in this case.
- For an n-point Gauss–Legendre rule on $[a, b]$,

  $$\int_a^b f(x)\, dx \approx \sum_{i=1}^{n} w_i f(x_i),$$

  with

  $$x_i = \frac{a+b}{2} + \frac{b-a}{2}\xi_i, \qquad w_i = \frac{b-a}{2}c_i,$$

where ξ_i and c_i are the standard nodes and weights on $[-1, 1]$. Where ξ_i and c_i are $\pm 1/\sqrt{3}$ and 1 respectively within the 2-point context.

Then for this problem, since $a = 0$, $b = 1$, and

$$f(x) = e^{\alpha x - x^2/2},$$

we get

$$\int_0^1 e^{\alpha x - x^2/2}\, dx \approx \sum_{i=1}^{20} w_i e^{\alpha x_i - x_i^2/2},$$

where

$$x_i = \frac{1}{2} + \frac{1}{2}\xi_i, \qquad w_i = \frac{1}{2}c_i.$$

Then solve

$$\alpha \sum_{i=1}^{20} w_i e^{\alpha x_i - x_i^2/2} = 1.$$

- In the code I map from $[0, 1]$ to $[-1, 1]$ then define $F(\alpha)$ as $\sum_{i=1}^{20} w_i e^{\alpha x_i - x_i^2/2} - 1$ and solve for $F(\alpha) = 0$
- I included a quick check and as you can see $4.440892098500626e - 16 \approx 0$

```
function solve_alpha_gauss_legendre

clc;
clear;

% standard 20-point Gauss-Legendre nodes on [-1,1]
xi = [ ...
    -0.993128599185095;
    -0.963971927277914;
    -0.912234428251326;
    -0.839116971822219;
    -0.746331906460151;
    -0.636053680726515;
    -0.510867001950827;
    -0.373706088715420;
    -0.227785851141645;
    -0.0765265211334973;
     0.0765265211334973;
```

```
        0.227785851141645;
        0.373706088715420;
        0.510867001950827;
        0.636053680726515;
        0.746331906460151;
        0.839116971822219;
        0.912234428251326;
        0.963971927277914;
        0.993128599185095 ];

    % standrd 20-point Gauss-Legendre weights on [-1,1]
    c = [ ...
        0.0176140071391521;
        0.0406014298003869;
        0.0626720483341091;
        0.0832767415767048;
        0.101930119817240;
        0.118194531961518;
        0.131688638449177;
        0.142096109318382;
        0.149172986472604;
        0.152753387130726;
        0.152753387130726;
        0.149172986472604;
        0.142096109318382;
        0.131688638449177;
        0.118194531961518;
        0.101930119817240;
        0.0832767415767048;
        0.0626720483341091;
        0.0406014298003869;
        0.0176140071391521 ];

    % the general interval [a,b]
    a = 0;
    b = 1;

    % Map nodes and weights from [-1,1] to [0,1]
    x = (a + b)/2 + (b - a)/2 * xi;
    w = (b - a)/2 * c;

```

```
    % we define F(alpha)
    F = @(alpha) alpha * sum(w .* exp(alpha .* x - x.^2 / 2)) - 1;

    % Solve F(alpha) = 0
    alpha_sol = fzero(F, [0 1]);

    fprintf('alpha = %.15f\n', alpha_sol);
    % check that F(alpha) = 0
    fprintf('Check: F(alpha) = %.15e\n', F(alpha_sol));

end
```

MATLAB Computation

```
alpha = 0.791853491334777
F(alpha) = 4.440892098500626e-16
```

Figure 1: Results

B.)

At time t the price of an asset is given by

$$P(t) = \int_t^T De^{-\rho(s-t)} \frac{u'(c(s))}{u'(c(t))} ds$$

where D is a time-invariant, deterministic dividend paid by the asset, and ρ measures the rate of impatience. Suppose the investor has a constant consumption stream, $c(s) = c(t)$, $\forall s \in [t, T]$. Assume that the scrap value of the asset is zero, that is, $P(T) = 0$.

1. Write down the closed-form solution for the price at time t. What is $P(0)$ when $D = 1$, $\rho = 4\%$ and $T = 100$? What is the limit of the asset's price as $T \to \infty$ when $D = 1$ and $\rho = 4\%$?

2. Find $\frac{dP}{dt}$ using Leibniz's rule. Together with the fact that $P(T) = 0$ you now have a first-order ODE.

3. Briefly explain Euler's method. With the ODE just obtained, use Euler's method to approximate the price of the asset over the period from $t = 0$ to $T = 100$. Use the values $D = 1$ and $\rho = 4\%$. Draw the graph of the approximated price together with the graph of the actual price.

1

With:

- $D = 1$
- $\rho = 4\%$
- $T = 100$

Because consumption is constant,

$$c(s) = c(t) \implies u'(c(s)) = u'(c(t))$$

so we have,

$$\frac{u'(c(s))}{u'(c(t))} = 1$$

The price formula simplifies immediately to

$$P(t) = \int_t^T De^{-\rho(s-t)}\, ds$$

And,

$$P(t) = D \int_t^T e^{-\rho(s-t)}\, ds$$

At this point integrate.

A clean substitution is:

$$x = s - t \implies ds = dx$$

When $s = t$, $x = 0$. so $s = T$, $x = T - t$.

So:

$$P(t) = D \int_0^{T-t} e^{-\rho x}\, dx$$

Integrate:

$$\int e^{-\rho x}\,dx = -\frac{1}{\rho}e^{-\rho x}$$

$$P(t) = D[\frac{-1}{\rho}e^{-\rho(T-t)} - \frac{-1}{\rho}e^{-\rho(0)}]$$

$$P(t) = D[\frac{-1}{\rho}e^{-\rho(T-t)} + \frac{1}{\rho}]$$

Hence:

$$P(t) = D\left[\frac{1-e^{-\rho(T-t)}}{\rho}\right]$$

So the closed-form solution is:

$$\boxed{P(t) = \frac{D}{\rho}\left(1-e^{-\rho(T-t)}\right)}$$

Step 1a: Find $P(0)$

Just plug in $t = 0$:

$$P(0) = \frac{D}{\rho}\left(1-e^{-\rho T}\right)$$

With $D = 1, \rho = 0.04, T = 100$:

$$P(0) = \frac{1}{0.04}\left(1-e^{-4}\right)$$

Since $\frac{1}{0.04} = 25$, we have

$$P(0) = 25(1-e^{-4})$$

Numerically, $e^{-4} \approx 0.0183$, so:

$$P(0) \approx 25(0.9817) \approx 24.54$$

So approximately:

$$\boxed{P(0) \approx 24.54}$$

Step 1b: Find the limit as $T \to \infty$

Use:

$$P(t) = \frac{D}{\rho}\left(1 - e^{-\rho(T-t)}\right)$$

If $\rho > 0$, then as $T \to \infty$:

$$e^{-\rho(T-t)} = e^{-\infty} \to 0$$

so:

$$\lim_{T\to\infty} P(t) = \frac{D}{\rho}$$

With $D = 1, \rho = 0.04$:

$$\boxed{\lim_{T\to\infty} P(t) = 25}$$

Converges close to $T = 100$.

2

The general Leibniz Integral Rule is:

$$\frac{d}{dt}\int_{a(t)}^{b(t)} f(s,t)\,ds = f(b(t),t)b'(t) - f(a(t),t)a'(t) + \int_{a(t)}^{b(t)} \frac{\partial}{\partial t} f(s,t)\,ds$$

In our case:

- $a(t) = t \implies a'(t) = 1$
- $b(t) = T \implies b'(t) = 0$
- $f(s,t) = De^{-\rho(s-t)}$

Applying the rule:

$$\frac{dP}{dt} = 0 - f(t,t)\cdot 1 + \int_t^T \frac{\partial}{\partial t}\left(De^{-\rho(s-t)}\right)\,ds$$

1. Boundary term:

$$f(t,t) = De^{-\rho(t-t)} = De^0 = D$$

So the contribution is $-D$.

2. **Partial derivative of the integrand:** Since $\frac{d}{dt}[-\rho(s-t)] = \rho$, we have:

$$\frac{\partial f}{\partial t} = \rho D e^{-\rho(s-t)}$$

3. **Assembly:**

$$\frac{dP}{dt} = -D + \int_t^T \rho D e^{-\rho(s-t)}\, ds$$

Factoring out ρ:

$$\frac{dP}{dt} = -D + \rho \int_t^T D e^{-\rho(s-t)}\, ds$$

Recognizing the original integral $P(t)$:

$$\boxed{\frac{dP}{dt} = \rho P(t) - D}$$

with the terminal condition:

$$\boxed{P(T) = 0}$$

The asset price changes over time from two opposing factors:

- Discounting future dividends makes them more valuable as they get closer
- The dividend stream is getting used up

3

From the previous analysis, the ODE is:

$$\frac{dP}{dt} = \rho P(t) - D, \qquad P(T) = 0$$

with the following parameters:

$$D = 1, \qquad \rho = 0.04, \qquad T = 100$$

Substituting these values into the expression, the specific ODE becomes:

$$P'(t) = 0.04P(t) - 1, \qquad P(100) = 0$$

Euler's Method

In one line Euler's method answers this: If I know the value of a function at one point, and I know the slope at that point how can I better estimate the actual function? The thought process is: look at where you currently, calculate the slope at that point, and take one small step using that slope. In our case we must go backward as we have the terminal value. The method starts from a known value and repeatedly uses the derivative to estimate the function value at the next grid point. Its accuracy improves as the step size becomes smaller.

So here, we define the function for the ODE:

$$f(t, P) = 0.04P - 1$$

Using the backward formula (stepping from n to $n-1$):

$$P_{n-1} = P_n - h\, f(t_n, P_n)$$

Substituting the specific form of f:

$$P_{n-1} = P_n - h(0.04P_n - 1)$$

Expanding the terms:

$$P_{n-1} = P_n - 0.04hP_n + h$$

$$\boxed{P_{n-1} = (1 - 0.04h)P_n + h}$$

```

clc;
clear;
close all;

% Parameters
D = 1;
rho = 0.04;
T = 100;

% number of steps
N = 100;                % increase N for better accuracy
h = T / N;

% time grid
```

```
t = linspace(0, T, N+1);

% Exact solution
P_exact = (D/rho) * (1 - exp(-rho * (T - t)));

% euler approximation
P_euler = zeros(1, N+1);

% Terminal condition: P(T) = 0
P_euler(N+1) = 0;

% Backward Euler stepping in time
for n = N+1:-1:2
    P_euler(n-1) = (1 - h * rho) * P_euler(n) + h * D;
end

% Plot
figure;
plot(t, P_exact, 'b-', 'LineWidth', 2);
hold on;
plot(t, P_euler, 'r--', 'LineWidth', 2);
xlabel('t');
ylabel('P(t)');
title('Exact and Euler Approximate Asset Price');
legend('Exact solution', 'Euler approximation', 'Location', 'best');
grid on;
```

MATLAB Computation

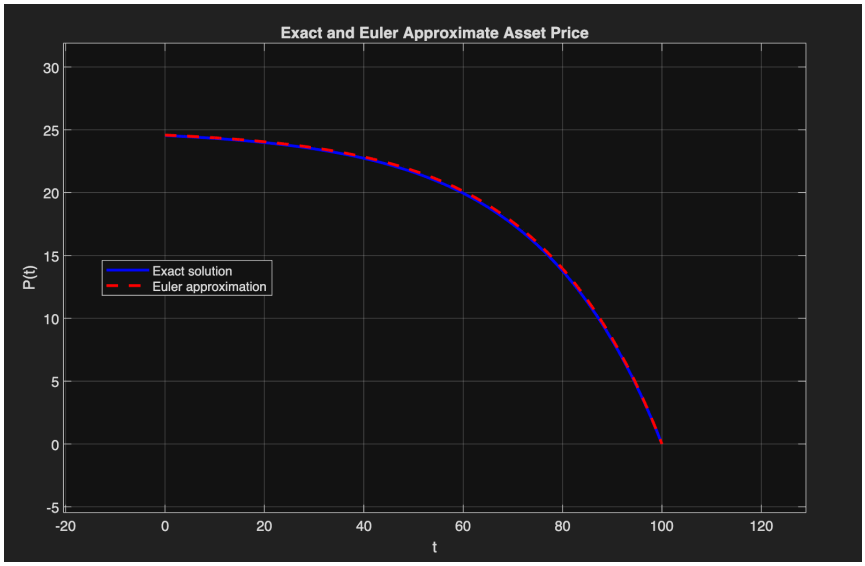

Figure 2: Results from Code

C.)

For each of the following standard interpolation schemes, construct the 15-node approximants for the function $f(x) = \exp(-x^2)$ on the interval $[-1, 1]$. In each case, briefly explain the interpolation method, and plot the approximation errors.

1. Naive polynomial interpolation. Briefly explain why naive polynomial interpolation can be problematic.
2. Chebychev polynomial approximation;
3. linear spline approximation; and
4. cubic spline approximation.

1. Naive Polynomial Interpolation

In this rather naive case we use the standard monomial-basis interpolation, $\{1, x, x^2, ..., x^n\}$. This naturally covers $\mathbb{R}$, and we have $n = 14$ in our case here.

- The $n + 1$ nodes are taken to be equally spaced.

- You then solve for coefficients $c_0, \ldots, c_{14}$ in

$$p_{14}(x) = c_0 + c_1 x + \cdots + c_{14} x^{14}$$

- s.t

$$p_{14}(x_i) = f(x_i), \qquad i = 1, \ldots, 15.$$

2 common issues can arise with this naive scheme: the Vandermonde matrix becomes badly conditioned as degree rises, becomes numerically unstable. The other issue is that equally spaced nodes can cause large oscillations of the Runge type, grows worse and worse as n rises.

2. Chebyshev polynomial interpolation

This method is used to avoid the instability of equally spaced polynomial interpolation (Runge type). The Chebyshev nodes on $[-1, 1]$ are:

$$x_j = \cos\left(\frac{(2j-1)\pi}{2n}\right), \quad j = 1, \ldots, n$$

In this case, $n = 15$. These nodes cluster near the endpoints, which is where naive equispaced interpolation tends to struggle. Note that $[-1, 1]$ works well in our case but we can map onto $[-1, 1]$ if our bounds are different.

The Chebyshev Basis The Chebyshev basis polynomials satisfy the following recurrence relation:

- $T_0(x) = 1$
- $T_1(x) = x$
- $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$

The approximation is then represented as:

$$p_{14}^{\text{cheb}}(x) = a_0 T_0(x) + a_1 T_1(x) + \cdots + a_{14} T_{14}(x)$$

For a fixed set of nodes and function values, changing the basis does not change the interpolating polynomial itself, only the coefficient representation. The stability improvement comes from the combination of the **Chebyshev basis** and **Chebyshev nodes**, which ensures the interpolation system is well-conditioned.

3. Linear Spline Approximation

A linear spline approximation is a piecewise linear segment on each interval $[x_i, x_{i+1}]$, continuous at the knots, but with jumps in the first derivative at the knots. We can write a linear spline with $n+1$ equally spaced nodes $x, x_1, x_2, ..., x_n$ as a linear combination of a hat-function basis $\phi_j(x)$. A linear spline can be expressed as

$$\hat{f} = \sum c_j \phi_j(x)$$

On each interval, only two adjacent hat functions are active, which ensures that interpolation is local.

For a system with $n+1 = 15$ nodes, you obtain 14 line segments joining the interpolation points exactly.

4. Cubic Spline Approximation

The principle is similar to linear and quadratic splines. Instead of the piece-wise polynomials being linear they are cubic. We guarantee this by adding a constraint onto quadratic splines. To ensure the splines are 'smooth' in x^3 not just the first derivative must be continuous at the node x_i but the second as well.

Implementation, errors

```
clear; clc; close all;

f = @(x) exp(-x.^2);

n = 15;                    % 15 nodes
deg = n - 1;               % degree 14 polynomial
xFine = linspace(-1,1,2000).';
yTrue = f(xFine); % calculates actual values

% 1) Naive polynomial interpolation
x_eq = linspace(-1,1,n).'; % using [-1,1] for all for easier
    comparison
y_eq = f(x_eq); % evaluates at nodes

% Solve in monomial basis using polyfit/polyval
p_naive = polyfit(x_eq, y_eq, deg);
y_naive = polyval(p_naive, xFine);
```

```
err_naive = y_naive - yTrue;

%2) Chebyshev polynomial interpolation
j = (1:n)';
x_cheb = cos((2*j - 1) * pi / (2*n));   % Chebyshev nodes on
    [-1,1]
y_cheb = f(x_cheb);

% build Chebyshev matrix T(:,k+1) = T_k(x_cheb), k=0,...,n-1
T = zeros(n,n); % creates matrix
T(:,1) = 1; % T_0(x) = 1
if n >= 2
    T(:,2) = x_cheb;
end % T_1(x) = x
for k = 3:n
    T(:,k) = 2*x_cheb .* T(:,k-1) - T(:,k-2);
end % full 15 x 15 matrix

% Solve for Chebyshev coefficients
a = T \ y_cheb; % solves T * a = y_cheb

% Evaluate on fine grid
Tf = zeros(length(xFine), n); % very large matrix
Tf(:,1) = 1;
if n >= 2
    Tf(:,2) = xFine;
end
for k = 3:n
    Tf(:,k) = 2*xFine .* Tf(:,k-1) - Tf(:,k-2);
end % true values

y_chebpoly = Tf * a;
err_cheb = y_chebpoly - yTrue;

% 3) Linear spline interpolation
y_linear = interp1(x_eq, y_eq, xFine, 'linear');
err_linear = y_linear - yTrue;

% 4) Cubic spline interpolation
% MATLAB spline() is standard cubic spline
y_cubic = spline(x_eq, y_eq, xFine);
```

```
err_cubic = y_cubic - yTrue;

% Plot the approximants
figure;
plot(xFine, yTrue, 'k-', 'LineWidth', 2); hold on;
plot(xFine, y_naive, '--', 'LineWidth', 1.3);
plot(xFine, y_chebpoly, '-', 'LineWidth', 1.3);
plot(xFine, y_linear, ':', 'LineWidth', 1.6);
plot(xFine, y_cubic, '-.', 'LineWidth', 1.4);
plot(x_eq, y_eq, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
legend('True f(x)=e^{-x^2}', 'Naive polynomial', 'Chebyshev polynomial', ...
        'Linear spline', 'Cubic spline', 'Nodes', 'Location', 'best');
title('15-node approximants on [-1,1]');
xlabel('x'); ylabel('Function value');
grid on;

%%Plot the errors
figure;
plot(xFine, err_naive, 'LineWidth', 1.3); hold on;
plot(xFine, err_cheb, 'LineWidth', 1.3);
plot(xFine, err_linear, 'LineWidth', 1.3);
plot(xFine, err_cubic, 'LineWidth', 1.3);
yline(0,'k-');
legend('Naive polynomial error', 'Chebyshev polynomial error', ...
        'Linear spline error', 'Cubic spline error', 'Location', 'best');
title('Interpolation errors: approximant minus true function');
xlabel('x'); ylabel('Error');
grid on;

% Display max absolute errors
fprintf('Max |error|, naive polynomial   = %.6e\n', max(abs(err_naive)));
fprintf('Max |error|, Chebyshev poly    = %.6e\n', max(abs(err_cheb)));
```

```
fprintf('Max |error|, linear spline      = %.6e\n', max(abs(
    err_linear)));
fprintf('Max |error|, cubic spline       = %.6e\n', max(abs(
    err_cubic)));
```

MATLAB Computation

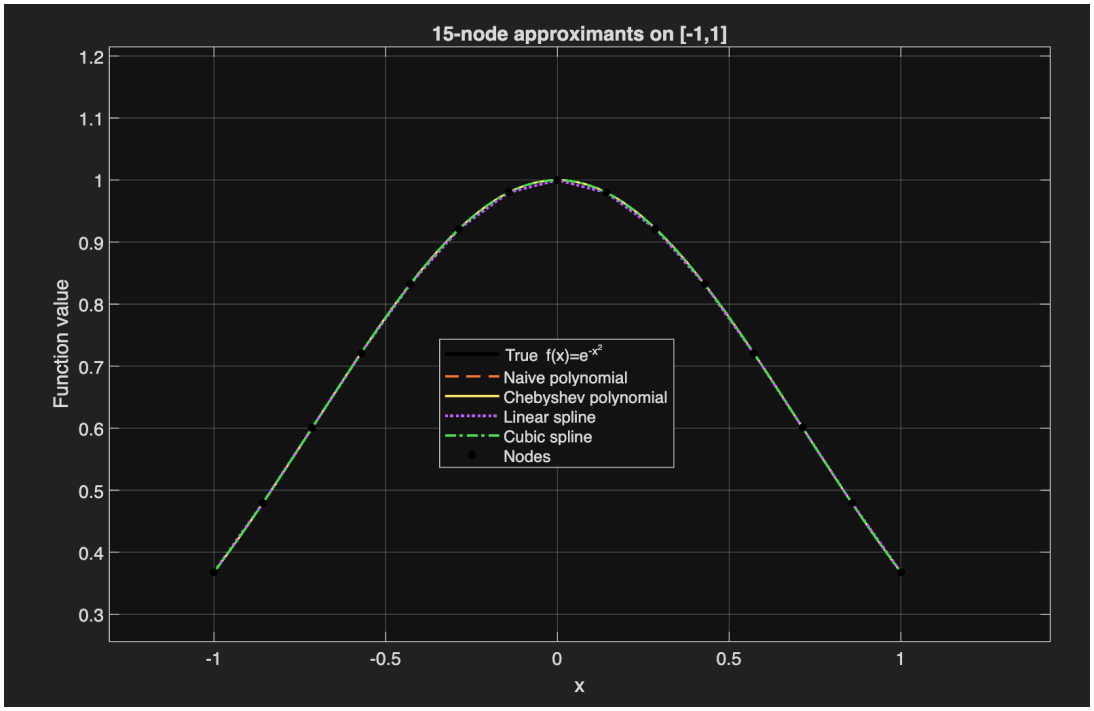

Figure 3: Interpolation Approximations

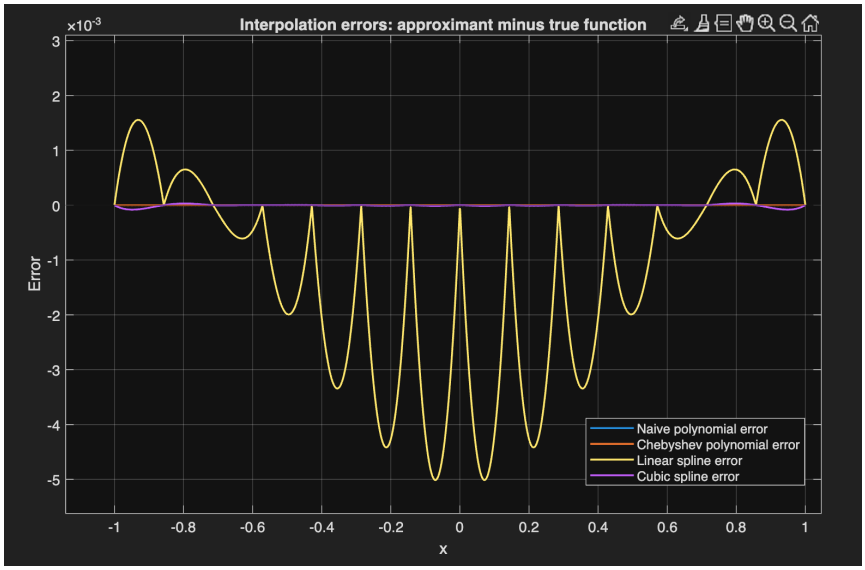

Figure 4: Errors: the approximations - true values

D.)

Consider the following consumption-savings problem of an agent. The agent has deterministic income flow $y(t)$, and chooses a consumption rate $c(t)$ to maximize discounted lifetime utility over a finite time horizon T:

$$\max_{c(t)} \int_0^T e^{-\rho t} u(c(t)) dt$$

$$\text{s.t. } \dot{w} = rw(t) + y(t) - c(t), \quad w(0) = 0$$

where w is wealth, r is the interest rate, and ρ is the rate-of-time preference. The solution to this problem can be expressed by solving the following system of ODEs:

$$\dot{c}(t) = -\frac{u'(c(t))}{u''(c(t))}(r - \rho) \tag{1}$$

$$\dot{w}(t) = rw(t) + y(t) - c(t) \tag{2}$$

where $w(0) = 0$, $w(T) = 0$.
Assume that the utility function is given by $u(c) = \frac{c^{1-\gamma}-1}{1-\gamma}$ and the deterministic income process by $y(t) = \frac{1}{1+e^{-\alpha t}}$. Assume the following parameter values: $\gamma = 2$,

$\alpha = 0.5$, $r = 0.04$ and $\rho = 0.04$. Use 10 collocation nodes in each function approximation.

1. Explain the collocation method and use it to solve the model. Plot consumption and wealth trajectories of your solution over time.
2. Repeat part (a) when the interest rate is $r = 0.03$ and when it is $r = 0.05$. Comment on the economics behind lifetime consumption and wealth for the three different interest rates.

Collocation is a method that approximates a simple shape of an unknown function and then forces the model to be true at a number of chosen points or functions.

1

Collocation method:

- Decide what class of approximating functions you will allow - Chebyshev polynomials, splines, etc. This turns an unknown function problem into an unknown coefficients problem.
- Plug approximate function into the equation, it will usually not satisfy $F(f, x) = 0$
- Force that error to be 0 at chosen nodes. Pick n points, called collocation nodes and require the residual to vanish there.

$$F(\hat{f}, x_i) = 0, \quad i = 1, 2, ..., n$$

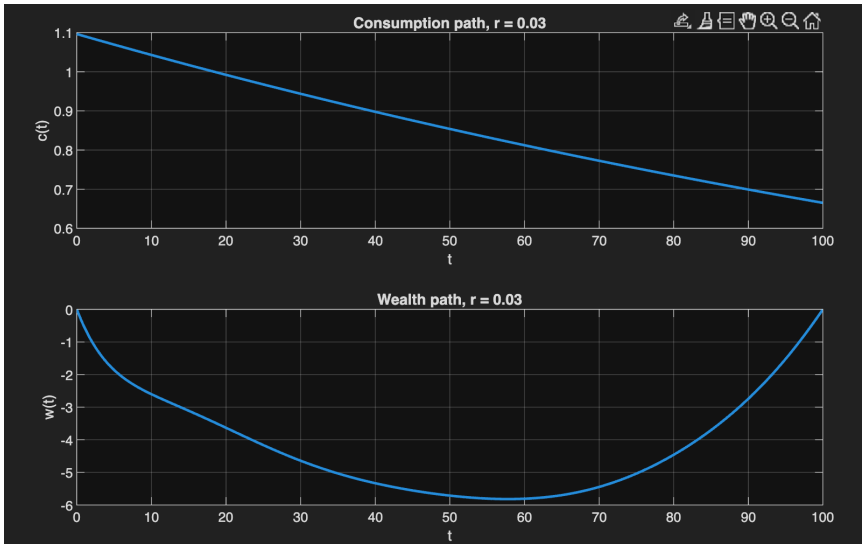

Figure 5: r = 0.03

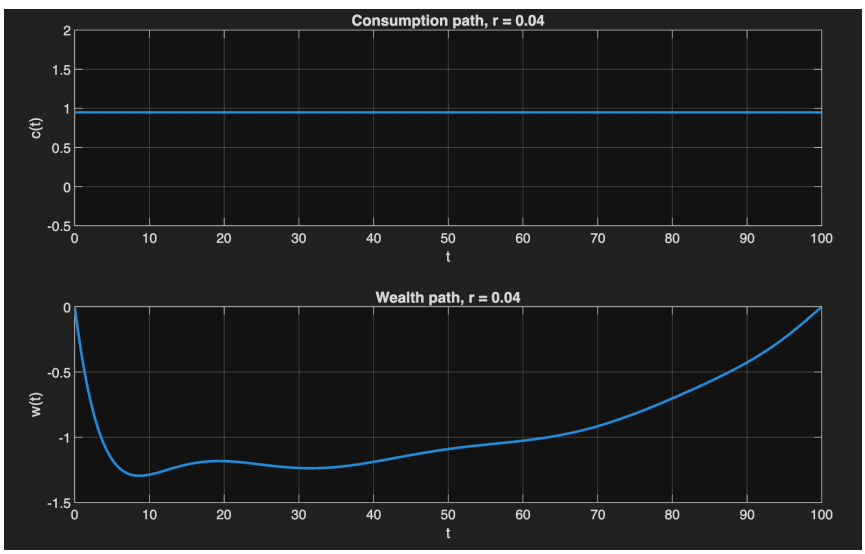

Figure 6: r = 0.04

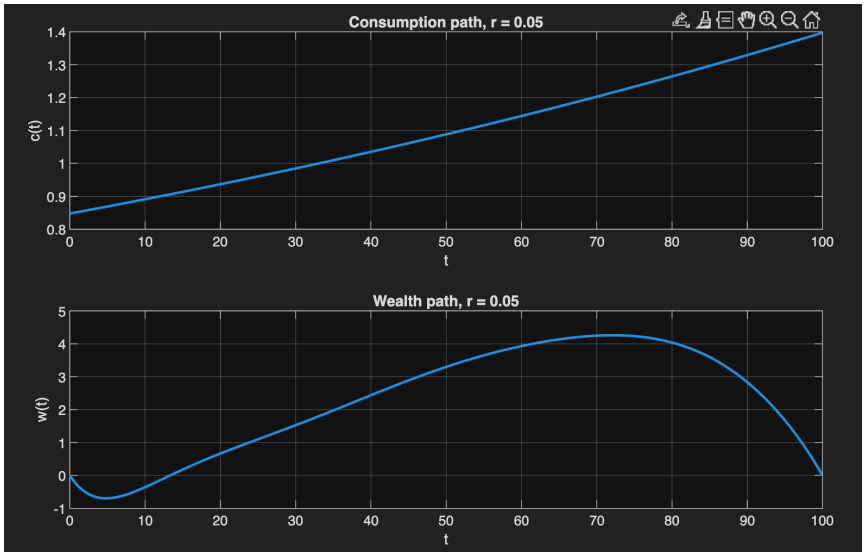

Figure 7: r = 0.05

2. Summary

The household's behavior across different interest rate regimes can be summarized as follows:

- **Impatient ($r = 0.03$):** The household is relatively impatient compared with the return on saving ($r < \rho$). Consequently, consumption is *front-loaded*, and wealth accumulation is lower over the life cycle.
- **Neutral ($r = 0.04$):** When the interest rate matches the discount rate ($r = \rho$), consumption is smoothed almost perfectly, resulting in a flat profile.
- **Incentivises ($r = 0.05$):** The higher return to saving ($r > \rho$) makes postponing consumption more attractive. Consumption rises over time (*back-loading*), and wealth levels are stronger earlier in life as the agent builds a larger nest egg.

Conclusion: The main economic result is that an increase in the interest rate r shifts optimal consumption toward the future and increases the incentive to save.